Práctica N°1: Inicializacion de OLED y perifericos en PIC32 MAX32

Índice

1. Objetivos **2**

2. Configuración del compilador **2**
2.1. Selección de la tarjeta (Board) 3
2.2. Selección del puerto COM 3

3. Descripción del código **4**
3.1. Código completo 4
3.2. Explicación paso a paso del programa (código completo) . . . 12

4. Carga y verificación del programa en la placa **16**
4.1. Proceso de carga del programa 16
4.2. Verificación del funcionamiento 17

1. Objetivos

Objetivo general Realizar una práctica de laboratorio con la placa PIC32 (MAX32) usando el entorno de desarrollo MPIDE y la placa Basic I/O Shield, para aprender a configurar el entorno, manejar entradas/salidas digitales, controlar una pantalla OLED mediante SPI por bit-bang y leer señales analógicas con el ADC.

Objetivos específicos

1. Configurar el entorno de desarrollo MPIDE (instalación, ajuste de la placa MAX32 y selección del puerto serie) y preparar el proyecto base para cargar el código provisto.

2. Verificar y configurar correctamente el mapeo de pines entre el `Basic I/O Shield` y la placa MAX32, asegurando que las definiciones de pines en el código coincidan con el cableado físico.

3. Implementar y comprobar el control de E/S digitales:

   - Uso de pines configurados como `INPUT` (interruptores y botones) y `OUTPUT` (LEDs).

4. Inicializar y controlar una pantalla OLED por SPI emulado (bit-banging):

   - Implementar rutinas de envío de comandos y datos (funciones de `oled_write_command`, `oled_write_data`, `oled_init`, `oled_set_pos`).
   - Mostrar cadenas usando la fuente 5x8 incluida; limpiar y actualizar líneas concretas de la pantalla.

5. Leer y presentar señales analógicas con el ADC:

   - Realizar lecturas periódicas de `A0` con `analogRead`, convertir el valor a miliVoltios (asumiendo referencia de 3.3 V si aplica) y mostrar tanto el conteo ADC como el valor en mV en la OLED.
   - Verificar la resolución (10-bit en el ejemplo) y documentar cualquier diferencia real de resolución/VR en la placa MAX32.

2. Configuración del compilador

En esta sección se describe el procedimiento para seleccionar correctamente la tarjeta de desarrollo (board) y el puerto de comunicación `COM` en el entorno de desarrollo, utilizando el sistema operativo Windows.

2.1. Selección de la tarjeta (Board)

Una vez abierto el entorno de desarrollo, siga los pasos a continuación:

1. Abra el menú de configuración del proyecto o del entorno (por ejemplo, `Project → Properties` en STM32CubeIDE o `Tools → Board` en Arduino IDE).

2. Seleccione la opción correspondiente a su tarjeta de desarrollo.

3. En caso de trabajar con una tarjeta STM32, el entorno detectará automáticamente el modelo al crear el proyecto; de no ser así, seleccione manualmente el microcontrolador `STM32F407VG` o la tarjeta `STM32F4DISCOVERY`.

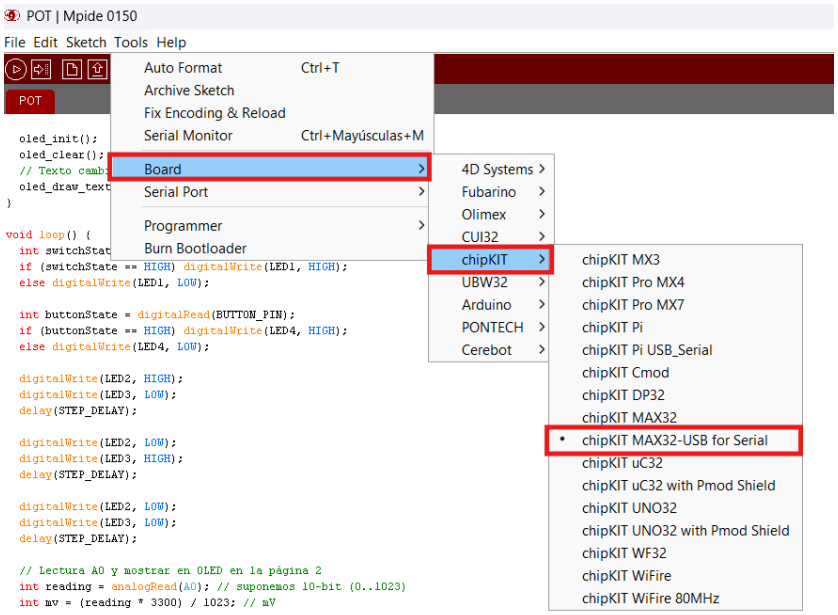


Figura 1: Selección de la tarjeta de desarrollo en el entorno de programación.

2.2. Selección del puerto COM

Para establecer la comunicación con la tarjeta a través del puerto serie (UART o USB), proceda de la siguiente manera:

1. Conecte la tarjeta al computador mediante el cable USB.

2. Abra el `Administrador de dispositivos` de Windows (`Inicio → Administrador de dispositivos → Puertos (COM y LPT)`).

3. Identifique el puerto `COM` asignado a la tarjeta (por ejemplo, `COM5`).

4. Regrese al entorno de desarrollo y seleccione el mismo puerto en el menú `Tools → Port` (Arduino IDE) o en la configuración de la sesión de depuración (STM32CubeIDE).

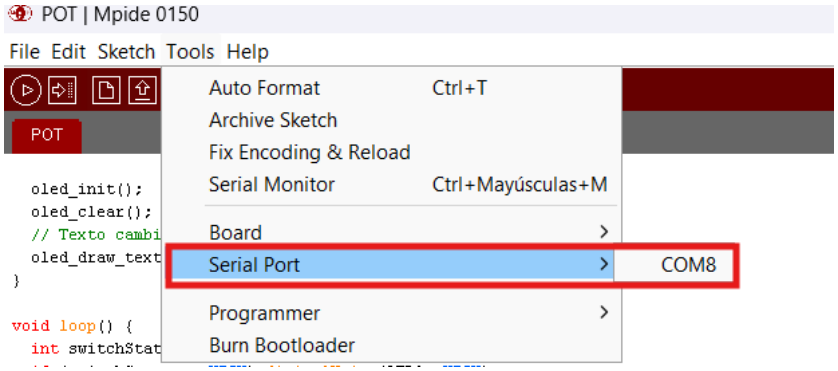


Figura 2: Selección del puerto COM correspondiente en el entorno de desarrollo.

Con estos pasos, la conexión entre el entorno y la tarjeta queda correctamente configurada para la compilación y carga del programa en Windows.

3. Descripción del código

3.1. Código completo

```
// ========== Config pines (exactamente los tuyos) ==========
#define LED1 77        // LED controlado por SWITCH_PIN (forzado por el switch)
#define LED2 76        // LED secuencia
#define LED3 75        // LED secuencia
#define LED4 74        // LED controlado por BUTTON_PIN (encendido mientras se pulsa)

#define SWITCH_PIN 79 // Switch de 2 posiciones: nivel 1 o 0 (forza LED1)
#define BUTTON_PIN 78 // Bot n moment neo: mientras est  pulsado -> LED4 ON

```

```
// OLED SPI pins (bit-bang)
#define OLED_RES 10   // RESET
#define OLED_DC  83   // D/C (0 = command, 1 = data)
#define OLED_SCLK 13  // SCLK
#define OLED_SDIN 11  // MOSI (SDIN)
// VDD = 82, VBAT = 84  -> ADVERTENCIA: no conectes VDD/VBAT a GPIO a menos que est s seguro.

#define STEP_DELAY 500  // ms secuencia

// ================= FONT 5x8 ASCII completo (0x20..0x7F) ==================
// Cada car cter ocupa 5 columnas; fuente est ndar 5x8 (adaptada de fuentes p blicas).
const uint8_t font5x8_custom[][5] = {
  {0x00,0x00,0x00,0x00,0x00}, // 0x20 ' '
  {0x00,0x00,0x5F,0x00,0x00}, // 0x21 '!'
  {0x00,0x07,0x00,0x07,0x00}, // 0x22 '"'
  {0x14,0x7F,0x14,0x7F,0x14}, // 0x23 '#'
  {0x24,0x2A,0x7F,0x2A,0x12}, // 0x24 '$'
  {0x23,0x13,0x08,0x64,0x62}, // 0x25 '%'
  {0x36,0x49,0x55,0x22,0x50}, // 0x26 '&'
  {0x00,0x05,0x03,0x00,0x00}, // 0x27 '''
  {0x00,0x1C,0x22,0x41,0x00}, // 0x28 '('
  {0x00,0x41,0x22,0x1C,0x00}, // 0x29 ')'
  {0x14,0x08,0x3E,0x08,0x14}, // 0x2A '*'
  {0x08,0x08,0x3E,0x08,0x08}, // 0x2B '+'
  {0x00,0x50,0x30,0x00,0x00}, // 0x2C ','
  {0x08,0x08,0x08,0x08,0x08}, // 0x2D '-'
  {0x00,0x60,0x60,0x00,0x00}, // 0x2E '.'
  {0x20,0x10,0x08,0x04,0x02}, // 0x2F '/'
  {0x3E,0x51,0x49,0x45,0x3E}, // 0x30 '0'
  {0x00,0x42,0x7F,0x40,0x00}, // 0x31 '1'
  {0x42,0x61,0x51,0x49,0x46}, // 0x32 '2'
  {0x21,0x41,0x45,0x4B,0x31}, // 0x33 '3'
  {0x18,0x14,0x12,0x7F,0x10}, // 0x34 '4'
  {0x27,0x45,0x45,0x45,0x39}, // 0x35 '5'
  {0x3C,0x4A,0x49,0x49,0x30}, // 0x36 '6'
  {0x01,0x71,0x09,0x05,0x03}, // 0x37 '7'
  {0x36,0x49,0x49,0x49,0x36}, // 0x38 '8'
  {0x06,0x49,0x49,0x29,0x1E}, // 0x39 '9'
  {0x00,0x36,0x36,0x00,0x00}, // 0x3A ':'
  {0x00,0x56,0x36,0x00,0x00}, // 0x3B ';'
```

```
50    {0x08,0x14,0x22,0x41,0x00}, // 0x3C '<'
51    {0x14,0x14,0x14,0x14,0x14}, // 0x3D '='
52    {0x00,0x41,0x22,0x14,0x08}, // 0x3E '>'
53    {0x02,0x01,0x51,0x09,0x06}, // 0x3F '?'
54    {0x32,0x49,0x79,0x41,0x3E}, // 0x40 '@'
55    {0x7E,0x11,0x11,0x11,0x7E}, // 0x41 'A'
56    {0x7F,0x49,0x49,0x49,0x36}, // 0x42 'B'
57    {0x3E,0x41,0x41,0x41,0x22}, // 0x43 'C'
58    {0x7F,0x41,0x41,0x22,0x1C}, // 0x44 'D'
59    {0x7F,0x49,0x49,0x49,0x41}, // 0x45 'E'
60    {0x7F,0x09,0x09,0x09,0x01}, // 0x46 'F'
61    {0x3E,0x41,0x49,0x49,0x7A}, // 0x47 'G'
62    {0x7F,0x08,0x08,0x08,0x7F}, // 0x48 'H'
63    {0x00,0x41,0x7F,0x41,0x00}, // 0x49 'I'
64    {0x20,0x40,0x41,0x3F,0x01}, // 0x4A 'J'
65    {0x7F,0x08,0x14,0x22,0x41}, // 0x4B 'K'
66    {0x7F,0x40,0x40,0x40,0x40}, // 0x4C 'L'
67    {0x7F,0x02,0x0C,0x02,0x7F}, // 0x4D 'M'
68    {0x7F,0x04,0x08,0x10,0x7F}, // 0x4E 'N'
69    {0x3E,0x41,0x41,0x41,0x3E}, // 0x4F 'O'
70    {0x7F,0x09,0x09,0x09,0x06}, // 0x50 'P'
71    {0x3E,0x41,0x51,0x21,0x5E}, // 0x51 'Q'
72    {0x7F,0x09,0x19,0x29,0x46}, // 0x52 'R'
73    {0x26,0x49,0x49,0x49,0x32}, // 0x53 'S'
74    {0x01,0x01,0x7F,0x01,0x01}, // 0x54 'T'
75    {0x3F,0x40,0x40,0x40,0x3F}, // 0x55 'U'
76    {0x1F,0x20,0x40,0x20,0x1F}, // 0x56 'V'
77    {0x3F,0x40,0x38,0x40,0x3F}, // 0x57 'W'
78    {0x63,0x14,0x08,0x14,0x63}, // 0x58 'X'
79    {0x07,0x08,0x70,0x08,0x07}, // 0x59 'Y'
80    {0x61,0x51,0x49,0x45,0x43}, // 0x5A 'Z'
81    {0x00,0x7F,0x41,0x41,0x00}, // 0x5B '['
82    {0x02,0x04,0x08,0x10,0x20}, // 0x5C '\'
83    {0x00,0x41,0x41,0x7F,0x00}, // 0x5D ']'
84    {0x04,0x02,0x01,0x02,0x04}, // 0x5E '^'
85    {0x40,0x40,0x40,0x40,0x40}, // 0x5F '_'
86    {0x00,0x01,0x02,0x04,0x00}, // 0x60 '`'
87    {0x20,0x54,0x54,0x54,0x78}, // 0x61 'a'
88    {0x7F,0x48,0x44,0x44,0x38}, // 0x62 'b'
89    {0x38,0x44,0x44,0x44,0x20}, // 0x63 'c'
90    {0x38,0x44,0x44,0x48,0x7F}, // 0x64 'd'
91    {0x38,0x54,0x54,0x54,0x18}, // 0x65 'e'
92    {0x08,0x7E,0x09,0x01,0x02}, // 0x66 'f'
```

```
  {0x0C,0x52,0x52,0x52,0x3E}, // 0x67 'g'
  {0x7F,0x08,0x04,0x04,0x78}, // 0x68 'h'
  {0x00,0x44,0x7D,0x40,0x00}, // 0x69 'i'
  {0x20,0x40,0x44,0x3D,0x00}, // 0x6A 'j'
  {0x7F,0x10,0x28,0x44,0x00}, // 0x6B 'k'
  {0x00,0x41,0x7F,0x40,0x00}, // 0x6C 'l'
  {0x7C,0x04,0x18,0x04,0x78}, // 0x6D 'm'
  {0x7C,0x08,0x04,0x04,0x78}, // 0x6E 'n'
  {0x38,0x44,0x44,0x44,0x38}, // 0x6F 'o'
  {0x7C,0x14,0x14,0x14,0x08}, // 0x70 'p'
  {0x08,0x14,0x14,0x18,0x7C}, // 0x71 'q'
  {0x7C,0x08,0x04,0x04,0x08}, // 0x72 'r'
  {0x48,0x54,0x54,0x54,0x20}, // 0x73 's'
  {0x04,0x3F,0x44,0x40,0x20}, // 0x74 't'
  {0x3C,0x40,0x40,0x20,0x7C}, // 0x75 'u'
  {0x1C,0x20,0x40,0x20,0x1C}, // 0x76 'v'
  {0x3C,0x40,0x30,0x40,0x3C}, // 0x77 'w'
  {0x44,0x28,0x10,0x28,0x44}, // 0x78 'x'
  {0x0C,0x50,0x50,0x50,0x3C}, // 0x79 'y'
  {0x44,0x64,0x54,0x4C,0x44}, // 0x7A 'z'
  {0x00,0x08,0x36,0x41,0x00}, // 0x7B '{'
  {0x00,0x00,0x7F,0x00,0x00}, // 0x7C '|'
  {0x00,0x41,0x36,0x08,0x00}, // 0x7D '}'
  {0x10,0x08,0x08,0x10,0x08}  // 0x7E '~' (approx)
};
// ==============

// Map char -> index in font5x8_custom (ASCII 0x20..0x7F)
int char_index(char c) {
  if (c >= 0x20 && c <= 0x7E) return (c - 0x20);
  return 0; // fallback: space
}

// == Rutinas SPI por bit-bang para OLED ==
void oled_pin_init() {
  pinMode(OLED_RES, OUTPUT);
  pinMode(OLED_DC, OUTPUT);
  pinMode(OLED_SCLK, OUTPUT);
  pinMode(OLED_SDIN, OUTPUT);
}

void spi_delay() {
```

```
135   asm volatile("nop"); asm volatile("nop");
136 }
137
138 void spi_write_byte(uint8_t b) {
139   for (int i = 7; i >= 0; --i) {
140     digitalWrite(OLED_SCLK, LOW);
141     if (b & (1 << i)) digitalWrite(OLED_SDIN, HIGH);
142     else digitalWrite(OLED_SDIN, LOW);
143     spi_delay();
144     digitalWrite(OLED_SCLK, HIGH);
145     spi_delay();
146   }
147   digitalWrite(OLED_SCLK, LOW);
148 }
149
150 void oled_write_command(uint8_t cmd) {
151   digitalWrite(OLED_DC, LOW);
152   spi_write_byte(cmd);
153 }
154
155 void oled_write_data(uint8_t data) {
156   digitalWrite(OLED_DC, HIGH);
157   spi_write_byte(data);
158 }
159
160 void oled_init() {
161   oled_pin_init();
162   digitalWrite(OLED_RES, LOW);
163   delay(10);
164   digitalWrite(OLED_RES, HIGH);
165   delay(10);
166
167   oled_write_command(0xAE);
168   oled_write_command(0xD5); oled_write_command(0x80);
169   oled_write_command(0xA8); oled_write_command(0x1F);
170   oled_write_command(0xD3); oled_write_command(0x00);
171   oled_write_command(0x40);
172   oled_write_command(0x8D); oled_write_command(0x14);
173   oled_write_command(0x20); oled_write_command(0x00);
174   oled_write_command(0xA1);
175   oled_write_command(0xC8);
176   oled_write_command(0xDA); oled_write_command(0x02);
177   oled_write_command(0x81); oled_write_command(0x8F);
```

```
178   oled_write_command(0xD9); oled_write_command(0xF1);
179   oled_write_command(0xDB); oled_write_command(0x40);
180   oled_write_command(0xA4);
181   oled_write_command(0xA6);
182   oled_write_command(0xAF);
183 }
184
185 void oled_set_pos(uint8_t col, uint8_t page) {
186   oled_write_command(0xB0 | page);
187   oled_write_command(0x00 | (col & 0x0F));
188   oled_write_command(0x10 | ((col >> 4) & 0x0F));
189 }
190
191 void oled_draw_text_simple(uint8_t x, uint8_t page,
      const char *s) {
192   uint8_t col = x;
193   while (*s && col < 128) {
194     int idx = char_index(*s);
195     const uint8_t *glyph = font5x8_custom[idx];
196     oled_set_pos(col, page);
197     for (int i = 0; i < 5; ++i) {
198       oled_write_data(glyph[i]);
199     }
200     oled_write_data(0x00); // espacio entre caracteres
201     col += 6;
202     s++;
203   }
204 }
205
206 void oled_clear() {
207   for (uint8_t page = 0; page < 4; ++page) {
208     oled_set_pos(0, page);
209     for (uint8_t col = 0; col < 128; ++col) {
210       oled_write_data(0x00);
211     }
212   }
213 }
214
215 void oled_clear_line(uint8_t page) {
216   oled_set_pos(0, page);
217   for (uint8_t col = 0; col < 128; ++col) {
218     oled_write_data(0x00);
219   }
```

```
220 }
221
222 char *int_to_str(int val, char *buf) {
223   char tmp[12];
224   int i = 0;
225   if (val == 0) {
226     buf[0] = '0';
227     buf[1] = '\0';
228     return buf;
229   }
230   while (val > 0 && i < 11) {
231     tmp[i++] = '0' + (val % 10);
232     val /= 10;
233   }
234   int j;
235   for (j = 0; j < i; ++j) buf[j] = tmp[i - 1 - j];
236   buf[j] = '\0';
237   return buf;
238 }
239
240 // ========== Setup y loop principales ==========
241 void setup() {
242   pinMode(LED1, OUTPUT);
243   pinMode(LED2, OUTPUT);
244   pinMode(LED3, OUTPUT);
245   pinMode(LED4, OUTPUT);
246
247   pinMode(SWITCH_PIN, INPUT);
248   pinMode(BUTTON_PIN, INPUT);
249
250   pinMode(A0, INPUT);
251
252   digitalWrite(LED1, LOW);
253   digitalWrite(LED2, LOW);
254   digitalWrite(LED3, LOW);
255   digitalWrite(LED4, LOW);
256
257   oled_init();
258   oled_clear();
259   // Texto cambiado:
260   oled_draw_text_simple(0, 0, "Lectura␣de␣Potenciometro"
        );
261 }
```

```
262
263 void loop() {
264   int switchState = digitalRead(SWITCH_PIN);
265   if (switchState == HIGH) digitalWrite(LED1, HIGH);
266   else digitalWrite(LED1, LOW);
267
268   int buttonState = digitalRead(BUTTON_PIN);
269   if (buttonState == HIGH) digitalWrite(LED4, HIGH);
270   else digitalWrite(LED4, LOW);
271
272   digitalWrite(LED2, HIGH);
273   digitalWrite(LED3, LOW);
274   delay(STEP_DELAY);
275
276   digitalWrite(LED2, LOW);
277   digitalWrite(LED3, HIGH);
278   delay(STEP_DELAY);
279
280   digitalWrite(LED2, LOW);
281   digitalWrite(LED3, LOW);
282   delay(STEP_DELAY);
283
284   // Lectura A0 y mostrar en OLED en la p gina 2
285   int reading = analogRead(A0); // suponemos 10-bit
        (0..1023)
286   int mv = (reading * 3300) / 1023; // mV
287
288   char buf[24];
289   char num1[12], num2[12];
290   int_to_str(reading, num1);
291   int_to_str(mv, num2);
292
293   int pos = 0;
294   for (int i = 0; num1[i] != '\0'; ++i) buf[pos++] =
        num1[i];
295   buf[pos++] = ' ';
296   for (int i = 0; num2[i] != '\0'; ++i) buf[pos++] =
        num2[i];
297   buf[pos++] = 'm';
298   buf[pos++] = 'V';
299   buf[pos] = '\0';
300
301   oled_clear_line(2);
```

```
302   oled_draw_text_simple(0, 2, buf);
303
304   delay(200);
305 }
```

3.2. Explicación paso a paso del programa (código completo)

```
1 /* pega aqu  exactamente el c digo que me diste (
     mant n identaci n) */
2 // ========== Config pines (exactamente los tuyos)
     ==========
3 // ...
4 // (omito por brevedad en este ejemplo; en el informe
     deja el c digo entero)
```

Listing 1: Código completo usado en la práctica (OLED bit-bang, lectura A0, 4 LEDs y botones).

A continuación explicamos cada bloque importante del código, por qué está ahí, riesgos comunes, y qué modificar para replicar o mejorar el programa.

1. Definiciones de pines y constantes

- `#define LED1 77 ...LED4 74`
  Mapeo físico de LEDs: cada macro representa un GPIO. Mantén tu esquema de pines consistente con el hardware.
  Consejo / reto: documenta en una tabla física (p. ej. `LED1 = D13`) y verifica con un multímetro antes de probar.

- `SWITCH_PIN, BUTTON_PIN`
  Entrada digital para switch (posiciones) y botón momentáneo. El código asume lógica `HIGH = activo`. Si tu switch usa pull-down o pull-up internos, ajusta con `pinMode(..., INPUT_PULLUP)` y/o invierte la condición.

- Pines OLED (bit-bang): `OLED_RES, OLED_DC, OLED_SCLK, OLED_SDIN`
  ADVERTENCIA ya en el código: VDD/VBAT no deben conectarse a GPIO sin el hardware adecuado.

- `STEP_DELAY` controla la velocidad de la secuencia de LEDs. Cambia para acelerar/ralentizar.

2. Fuente 5×8 (tabla `font5x8_custom`)

- Es una matriz `const uint8_t [...] [5]` con los 95 glifos ASCII (0x20..0x7E). Cada carácter ocupa 5 columnas (bytes).
- Cada byte representa 8 píxeles verticales (bit 0 → pixel inferior o superior según convención del OLED). El código asume la orientación correcta de la pantalla.
- **Optimización:** si usas un compilador AVR/Arduino, coloca la matriz en memoria de programa (`PROGMEM`) para ahorrar SRAM.

3. Función `char_index`

- Traduce un `char` ASCII válido (0x20..0x7E) al índice de la matriz de fuente. Si el carácter no está en el rango devuelve el índice de espacio.
- **Precaución:** si tu entrada puede ser UTF-8, debes convertir antes a ASCII o implementar un decoder.

4. Rutinas SPI por bit-bang para la OLED

- `oled_pin_init()` — configura los GPIOs usados por el bit-bang como salidas.
- `spi_delay()` — pequeñas instrucciones `nop` para garantizar tiempos; útil si la MCU es muy rápida.
- `spi_write_byte(uint8_t b)` — envía 8 bits MSB primero: pone SCLK bajo, escribe MOSI, hace `nop`, sube SCLK, etc. Al final deja SCLK en LOW.
- `oled_write_command` / `oled_write_data` — controlan la línea D/C (comando vs dato) y llaman a `spi_write_byte`.

5. Inicialización de la OLED (`oled_init`)

- Resetea la pantalla con `OLED_RES` (LOW → HIGH).
- Envía una secuencia de comandos estándar (ej.: `0xAE` = display off, `0xAF` = display on, ajustes de multiplex, contraste, etc.). Son comandos comunes para pantallas tipo SSD13xx.

6. Posicionamiento y dibujo de texto

- `oled_set_pos(col, page)` — establece la columna y la página (banco vertical de 8 píxeles) para próximas escrituras.
- `oled_draw_text_simple(x, page, s)` — recorre la cadena y, por cada carácter:
  1. obtiene el índice con `char_index`,
  2. carga el puntero al glifo,
  3. llama a `oled_set_pos` y envía 5 bytes (columnas) + 1 byte de espacio.

7. Borrado de pantalla y líneas

- `oled_clear()` — recorre todas las páginas (en este código 4 páginas → altura 32 px) y pone 0x00 en todas las columnas.
- `oled_clear_line(page)` — borra solo una página, útil para actualizar una línea sin tocar el resto.

8. Conversión entera a cadena (`int_to_str`)

- Convierte un entero positivo a cadena decimal. Maneja `0` correctamente; no maneja negativos.

9. `setup()` — inicializaciones principales

- Configura pines de LEDs como `OUTPUT` y botones como `INPUT`. Considera `INPUT_PULLUP` si no usas resistencias externas.
- Inicializa ADC: el código usa `pinMode(A0, INPUT)` pero la lectura real se hace con `analogRead(A0)`; asegúrate que la placa soporta `A0` así nombrado.
- Inicializa la OLED y escribe un título.

10. `loop()` — comportamiento principal

1. Leer `SWITCH_PIN` y forzar `LED1` según estado.
   Nota: la lógica asume `HIGH = activo`; si tu switch usa pullups invierte la condición.

2. Leer `BUTTON_PIN` y mantener `LED4` activo mientras el botón esté pulsado.

3. Secuencia de `LED2` y `LED3`: alternancia con `delay(STEP_DELAY)` (bloqueante).

4. Lectura analógica: `analogRead(A0)` supone resolución de 10 bits (0..1023). Convierte a mV con formula: `mv = (reading * 3300) / 1023`.
   Precisión: 3300 mV asume referencia de 3.3V. Si la referencia es diferente corrige la constante.

5. Construcción de cadena `buf` con lectura y mV, borrado de la línea 2 y dibujado del texto.

6. `delay(200)` al final: ciclo totalmente bloqueante.

11. Problemas comunes y soluciones rápidas

- **Pantalla en blanco:** revisa conexiones, VCC/GND, nivel lógico (3.3V vs 5V), y secuencia de reset. Comprueba que los comandos enviados coinciden con el controlador.
- **Lecturas ADC erráticas:** añade un pequeño capacitor y usa `analogRead` repetido + media móvil. Usa `INPUT_PULLUP` si el potenciómetro no está referenciado.
- **Flashing o parpadeo:** evita `oled_clear()` completa en cada ciclo. Usa `oled_clear_line()` o buffering.
- **Botón rebotes:** implementa debounce (software) o hardware (capacitor + resistor).

13. Snippets útiles

```
bool readButtonDebounced(uint8_t pin) {
  static uint32_t lastTime[2] = {0,0};
  static bool lastState[2] = {LOW, LOW};
  int idx = (pin==BUTTON_PIN) ? 0 : 1;
  bool cur = digitalRead(pin);
  uint32_t t = millis();
  if (cur != lastState[idx]) {
    lastTime[idx] = t;
    lastState[idx] = cur;
  } else if (t - lastTime[idx] > 50) { // 50 ms debounce
```

```
    return cur;
  }
  return lastState[idx];
}
```

```
uint32_t lastStep = 0;
int step = 0;
void loop_non_blocking() {
  if (millis() - lastStep >= STEP_DELAY) {
    lastStep = millis();
    step = (step + 1) % 3;
    if (step==0){ digitalWrite(LED2, HIGH); digitalWrite
       (LED3, LOW); }
    if (step==1){ digitalWrite(LED2, LOW);  digitalWrite
       (LED3, HIGH); }
    if (step==2){ digitalWrite(LED2, LOW);  digitalWrite
       (LED3, LOW); }
  }
  // resto de lecturas no bloqueantes...
}
```

4. Carga y verificación del programa en la placa

Una vez compilado correctamente el proyecto, el siguiente paso es cargar el código en la placa **PIC32** para verificar su funcionamiento. El entorno **MPLAB X IDE** permite realizar este proceso de forma directa a través del programador **PICkit 3/4** o mediante el **Debugger integrado**.

4.1. Proceso de carga del programa

1. Conectar la placa **PIC32** al computador mediante el cable USB, verificando que el programador sea reconocido por el entorno.

2. En la barra de herramientas de **MPLAB X IDE**, seleccionar el botón **Make and Program Device Main Project** (icono del microcontrolador con una flecha verde). Esto compila y carga automáticamente el programa.

3. Observar en la consola de salida que la programación se complete sin errores.

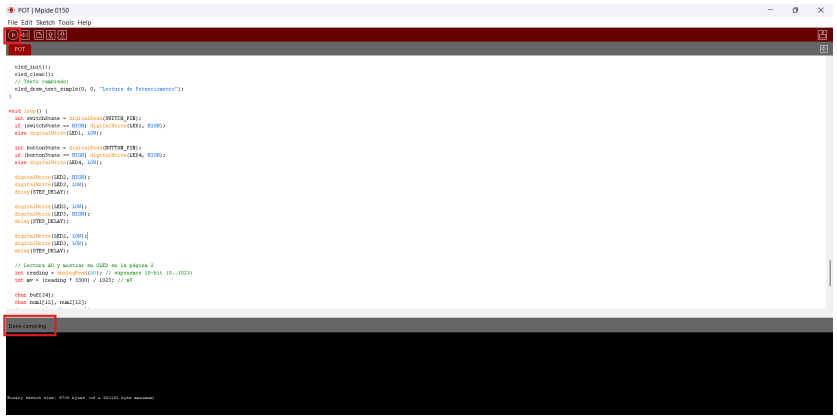


Figura 3: Compilación exitosa del proyecto en MPLAB X IDE.



Figura 4: Proceso de programación del microcontrolador PIC32.

4.2. Verificación del funcionamiento

Una vez finalizada la carga, el programa comienza a ejecutarse automáticamente. Es posible verificar su operación observando los resultados en la placa o por el monitor serie (según la funcionalidad implementada). Si se utilizan indicadores visuales como **LEDs** o mensajes por **UART**, deben reflejar el comportamiento esperado.

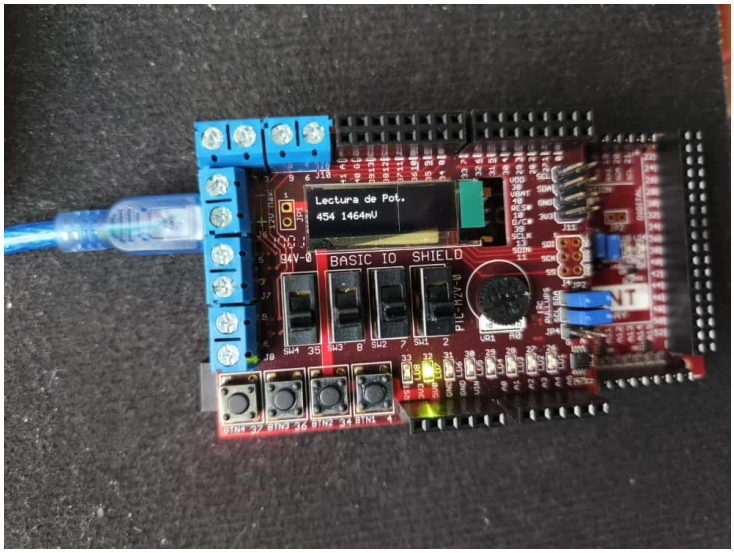


Figura 5: Verificación del funcionamiento del programa en la placa PIC32.